

```
[&os, c](const string &s) { os << s << c; });
```

可以很容易地编写一个函数，完成相同的工作：

```
ostream &print(ostream &os, const string &s, char c)
{
    return os << s << c;
}
```

但是，不能直接用 `bind` 来代替对 `os` 的捕获：

```
// 错误：不能拷贝 os
for_each(words.begin(), words.end(), bind(print, os, _1, ' '));
```

原因在于 `bind` 拷贝其参数，而我们不能拷贝一个 `ostream`。如果我们希望传递给 `bind` 一个对象而又不拷贝它，就必须使用标准库 **ref** 函数：

```
for_each(words.begin(), words.end(),
         bind(print, ref(os), _1, ' '));
```

函数 `ref` 返回一个对象，包含给定的引用，此对象是可以拷贝的。标准库中还有一个 **cref** 函数，生成一个保存 `const` 引用的类。与 `bind` 一样，函数 `ref` 和 `cref` 也定义在头文件 `functional` 中。

向后兼容：参数绑定

401

旧版本 C++ 提供的绑定函数参数的语言特性限制更多，也更复杂。标准库定义了两个分别名为 `bind1st` 和 `bind2nd` 的函数。类似 `bind`，这两个函数接受一个函数作为参数，生成一个新的可调用对象，该对象调用给定函数，并将绑定的参数传递给它。但是，这些函数分别只能绑定第一个或第二个参数。由于这些函数局限太强，在新标准中已被弃用（`deprecated`）。所谓被弃用的特性就是在新版本中不再支持的特性。新的 C++ 程序应该使用 `bind`。

10.3.4 节练习

练习 10.22：重写统计长度小于等于 6 的单词数量的程序，使用函数代替 `lambda`。

练习 10.23：`bind` 接受几个参数？

练习 10.24：给定一个 `string`，使用 `bind` 和 `check_size` 在一个 `int` 的 `vector` 中查找第一个大于 `string` 长度的值。

练习 10.25：在 10.3.2 节（第 349 页）的练习中，编写了一个使用 `partition` 的 `biggies` 版本。使用 `check_size` 和 `bind` 重写此函数。

10.4 再探迭代器

除了为每个容器定义的迭代器之外，标准库在头文件 `iterator` 中还定义了额外几种迭代器。这些迭代器包括以下几种。

- 插入迭代器（insert iterator）：这些迭代器被绑定到一个容器上，可用来向容器插入元素。
- 流迭代器（stream iterator）：这些迭代器被绑定到输入或输出流上，可用来遍历所

关联的 IO 流。

- **反向迭代器 (reverse iterator)**：这些迭代器向后而不是向前移动。除了 `forward_list` 之外的标准库容器都有反向迭代器。
- **移动迭代器 (move iterator)**：这些专用的迭代器不是拷贝其中的元素，而是移动它们。我们将在 13.6.2 节 (第 480 页) 介绍移动迭代器。



10.4.1 插入迭代器

插入器是一种迭代器适配器 (参见 9.6 节, 第 329 页)，它接受一个容器，生成一个迭代器，能实现向给定容器添加元素。当我们通过一个插入迭代器进行赋值时，该迭代器调用容器操作来向给定容器的指定位置插入一个元素。表 10.2 列出了这种迭代器支持的操作。

表 10.2: 插入迭代器操作	
<code>it = t</code>	在 <code>it</code> 指定的当前位置插入值 <code>t</code> 。假定 <code>c</code> 是 <code>it</code> 绑定的容器，依赖于插入迭代器的不同种类，此赋值会分别调用 <code>c.push_back(t)</code> 、 <code>c.push_front(t)</code> 或 <code>c.insert(t,p)</code> ，其中 <code>p</code> 为传递给 <code>inserter</code> 的迭代器位置
<code>*it, ++it, it++</code>	这些操作虽然存在，但不会对 <code>it</code> 做任何事情。每个操作都返回 <code>it</code>

402

插入器有三种类型，差异在于元素插入的位置：

- **back_inserter** (参见 10.2.2 节, 第 341 页) 创建一个使用 `push_back` 的迭代器。
- **front_inserter** 创建一个使用 `push_front` 的迭代器。
- **inserter** 创建一个使用 `insert` 的迭代器。此函数接受第二个参数，这个参数必须是一个指向给定容器的迭代器。元素将被插入到给定迭代器所表示的元素之前。



只有在容器支持 `push_front` 的情况下，我们才可以使用 `front_inserter`。类似的，只有在容器支持 `push_back` 的情况下，我们才能使用 `back_inserter`。

理解插入器的工作过程是很重要的：当调用 `inserter(c, iter)` 时，我们得到一个迭代器，接下来使用它时，会将元素插入到 `iter` 原来所指向的元素之前的位置。即，如果 `it` 是由 `inserter` 生成的迭代器，则下面这样的赋值语句

```
*it = val;
```

其效果与下面代码一样

```
it = c.insert(it, val); // it 指向新加入的元素
++it; // 递增 it 使它指向原来的元素
```

`front_inserter` 生成的迭代器的行为与 `inserter` 生成的迭代器完全不一样。当我们使用 `front_inserter` 时，元素总是插入到容器第一个元素之前。即使我们传递给 `inserter` 的位置原来指向第一个元素，只要我们在此元素之前插入一个新元素，此元素就不再是容器的首元素了：

```
list<int> lst = {1,2,3,4};
list<int> lst2, lst3; // 空 list
```

```
// 拷贝完成之后, lst2 包含 4 3 2 1
copy(lst.cbegin(), lst.cend(), front_inserter(lst2));
// 拷贝完成之后, lst3 包含 1 2 3 4
copy(lst.cbegin(), lst.cend(), inserter(lst3, lst3.begin()));
```

当调用 `front_inserter(c)` 时, 我们得到一个插入迭代器, 接下来会调用 `push_front`。当每个元素被插入到容器 `c` 中时, 它变为 `c` 的新的首元素。因此, `front_inserter` 生成的迭代器会将插入的元素序列的顺序颠倒过来, 而 `inserter` 和 `back_inserter` 则不会。

10.4.1 节练习

403

练习 10.26: 解释三种插入迭代器的不同之处。

练习 10.27: 除了 `unique` (参见 10.2.3 节, 第 343 页) 之外, 标准库还定义了名为 `unique_copy` 的函数, 它接受第三个迭代器, 表示拷贝不重复元素的目的位置。编写一个程序, 使用 `unique_copy` 将一个 `vector` 中不重复的元素拷贝到一个初始为空的 `list` 中。

练习 10.28: 一个 `vector` 中保存 1 到 9, 将其拷贝到三个其他容器中。分别使用 `inserter`、`back_inserter` 和 `front_inserter` 将元素添加到三个容器中。对每种 `inserter`, 估计输出序列是怎样的, 运行程序验证你的估计是否正确。

10.4.2 iostream 迭代器



虽然 `iostream` 类型不是容器, 但标准库定义了可以用于这些 IO 类型对象的迭代器 (参见 8.1 节, 第 278 页)。`istream_iterator` (参见表 10.3) 读取输入流, `ostream_iterator` (参见表 10.4 节, 第 361 页) 向一个输出流写数据。这些迭代器将它们对应的流当作一个特定类型的元素序列来处理。通过使用流迭代器, 我们可以用泛型算法从流对象读取数据以及向其写入数据。

istream_iterator 操作

当创建一个流迭代器时, 必须指定迭代器将要读写的对象类型。一个 `istream_iterator` 使用 `>>` 来读取流。因此, `istream_iterator` 要读取的类型必须定义了输入运算符。当创建一个 `istream_iterator` 时, 我们可以将它绑定到一个流。当然, 我们还可以默认初始化迭代器, 这样就创建了一个可以当作尾后值使用的迭代器。

```
istream_iterator<int> int_it(cin);    // 从 cin 读取 int
istream_iterator<int> int_eof;        // 尾后迭代器
ifstream in("afile");
istream_iterator<string> str_it(in);  // 从 "afile" 读取字符串
```

下面是一个用 `istream_iterator` 从标准输入读取数据, 存入一个 `vector` 的例子:

```
istream_iterator<int> in_iter(cin);    // 从 cin 读取 int
istream_iterator<int> eof;             // istream 尾后迭代器
while (in_iter != eof)                 // 当有数据可供读取时
    // 后置递增运算读取流, 返回迭代器的旧值
    // 解引用迭代器, 获得从流读取的前一个值
    vec.push_back(*in_iter++);
```

此循环从 cin 读取 int 值，保存在 vec 中。在每个循环步中，循环体代码检查 in_iter 是否等于 eof。eof 被定义为空的 istream_iterator，从而可以当作尾后迭代器来使用。对于一个绑定到流的迭代器，一旦其关联的流遇到文件尾或遇到 IO 错误，迭代器的值就与尾后迭代器相等。

404

此程序最困难的部分是传递给 push_back 的参数，其中用到了解引用运算符和后置递增运算符。该表达式的计算过程与我们之前写过的其他结合解引用和后置递增运算的表达式一样（参见 4.5 节，第 131 页）。后置递增运算会从流中读取下一个值，向前推进，但返回的是迭代器的旧值。迭代器的旧值包含了从流中读取的前一个值，对迭代器进行解引用就能获得此值。

我们可以将程序重写为如下形式，这体现了 istream_iterator 更有用的地方：

```
istream_iterator<int> in_iter(cin), eof; // 从 cin 读取 int
vector<int> vec(in_iter, eof); // 从迭代器范围构造 vec
```

本例中我们用一对表示元素范围的迭代器来构造 vec。这两个迭代器是 istream_iterator，这意味着元素范围是通过从关联的流中读取数据获得的。这个构造函数从 cin 中读取数据，直至遇到文件尾或者遇到一个不是 int 的数据为止。从流中读取的数据被用来构造 vec。

表 10.3: istream_iterator 操作

istream_iterator<T> in(is);	in 从输入流 is 读取类型为 T 的值
istream_iterator<T> end;	读取类型为 T 的值的 istream_iterator 迭代器，表示尾后位置
in1 == in2	in1 和 in2 必须读取相同类型。如果它们都是尾后迭代器，或绑定到相同的输入，则两者相等
in1 != in2	
*in	返回从流中读取的值
in->mem	与 (*in).mem 的含义相同
++in, in++	使用元素类型所定义的>>运算符从输入流中读取下一个值。与以往一样，前置版本返回一个指向递增后迭代器的引用，后置版本返回旧值

使用算法操作流迭代器

由于算法使用迭代器操作来处理数据，而流迭代器又至少支持某些迭代器操作，因此我们至少可以用某些算法来操作流迭代器。我们在 10.5.1 节（第 365 页）会看到如何分辨哪些算法可以用于流迭代器。下面是一个例子，我们可以用一对 istream_iterator 来调用 accumulate：

```
istream_iterator<int> in(cin), eof;
cout << accumulate(in, eof, 0) << endl;
```

此调用会计算出从标准输入读取的值的和。如果输入为：

23 109 45 89 6 34 12 90 34 23 56 23 8 89 23

则输出为 664。

405

~~istream_iterator 允许使用懒惰求值~~

当我们将一个 istream_iterator 绑定到一个流时，标准库并不保证迭代器立即从流读取数据。具体实现可以推迟从流中读取数据，直到我们使用迭代器时才真正读取。标

准库中的实现所保证的是，在我们第一次解引用迭代器之前，从流中读取数据的操作已经完成了。对于大多数程序来说，立即读取还是推迟读取没什么差别。但是，如果我们创建了一个 `istream_iterator`，没有使用就销毁了，或者我们正在从两个不同的对象同步读取同一个流，那么何时读取可能就很重要了。

ostream_iterator 操作

我们可以对任何具有输出运算符 (`<<`运算符) 的类型定义 `ostream_iterator`。当创建一个 `ostream_iterator` 时，我们可以提供 (可选的) 第二参数，它是一个字符串，在输出每个元素后都会打印此字符串。此字符串必须是一个 C 风格字符串 (即，一个字符串字面常量或者一个指向以空字符结尾的字符数组的指针)。必须将 `ostream_iterator` 绑定到一个指定的流，不允许空的或表示尾后位置的 `ostream_iterator`。

表 10.4: ostream_iterator 操作	
<code>ostream_iterator<T> out(os);</code>	<code>out</code> 将类型为 <code>T</code> 的值写到输出流 <code>os</code> 中
<code>ostream_iterator<T> out(os,d);</code>	<code>out</code> 将类型为 <code>T</code> 的值写到输出流 <code>os</code> 中，每个值后面都输出一个 <code>d</code> 。 <code>d</code> 指向一个空字符结尾的字符数组
<code>out = val</code>	用 <code><<</code> 运算符将 <code>val</code> 写入到 <code>out</code> 所绑定的 <code>ostream</code> 中。 <code>val</code> 的类型必须与 <code>out</code> 可写的类型兼容
<code>*out, ++out, out++</code>	这些运算符是存在的，但不 <code>out</code> 做任何事情。每个运算符都返回 <code>out</code>

我们可以用 `ostream_iterator` 来输出值的序列：

```
ostream_iterator<int> out_iter(cout, " ");
for (auto e : vec)
    *out_iter++ = e; // 赋值语句实际上将元素写到 cout
cout << endl;
```

此程序将 `vec` 中的每个元素写到 `cout`，每个元素后加一个空格。每次向 `out_iter` 赋值时，写操作就会被提交。

值得注意的是，当我们向 `out_iter` 赋值时，可以忽略解引用和递增运算。即，循环可以重写成下面的样子：

```
for (auto e : vec)
    out_iter = e; // 赋值语句将元素写到 cout
cout << endl;
```

运算符 `*`和`++`实际上对 `ostream_iterator` 对象不做任何事情，因此忽略它们对我们的程序没有任何影响。但是，推荐第一种形式。在这种写法中，流迭代器的使用与其他迭代器的使用保持一致。如果想将此循环改为操作其他迭代器类型，修改起来非常容易。而且，对于读者来说，此循环的行为也更为清晰。

可以通过调用 `copy` 来打印 `vec` 中的元素，这比编写循环更为简单：

```
copy(vec.begin(), vec.end(), out_iter);
cout << endl;
```


使用流迭代器处理类类型

我们可以为任何定义了输入运算符(>>)的类型创建 `istream_iterator` 对象。类似的,只要类型有输出运算符(<<),我们就可以为其定义 `ostream_iterator`。由于 `Sales_item` 既有输入运算符也有输出运算符,因此可以使用 IO 迭代器重写 1.6 节(第 21 页)中的书店程序:

```
istream_iterator<Sales_item> item_iter(cin), eof;
ostream_iterator<Sales_item> out_iter(cout, "\n");
// 将第一笔交易记录存在 sum 中,并读取下一条记录
Sales_item sum = *item_iter++;
while (item_iter != eof) {
    // 如果当前交易记录(存在 item_iter 中)有着相同的 ISBN 号
    if (item_iter->isbn() == sum.isbn())
        sum += *item_iter++;    // 将其加到 sum 上并读取下一条记录
    else {
        out_iter = sum;        // 输出 sum 当前值
        sum = *item_iter++;    // 读取下一条记录
    }
}
out_iter = sum;                // 记得打印最后一组记录的和
```

此程序使用 `item_iter` 从 `cin` 读取 `Sales_item` 交易记录,并将和写入 `cout`,每个结果后面都跟一个换行符。定义了自己的迭代器后,我们就可以用 `item_iter` 读取第一条交易记录,用它的值来初始化 `sum`:

```
// 将第一条交易记录保存在 sum 中,并读取下一条记录
Sales_item sum = *item_iter++;
```

此处,我们对 `item_iter` 执行后置递增操作,对结果进行解引用操作。这个表达式读取下一条交易记录,并用之前保存在 `item_iter` 中的值来初始化 `sum`。

`while` 循环会反复执行,直至在 `cin` 上遇到文件尾为止。在 `while` 循环体中,我们检查 `sum` 与刚刚读入的记录是否对应同一本书。如果两者的 ISBN 不同,我们将 `sum` 赋予 `out_iter`,这将会打印 `sum` 的当前值,并接着打印一个换行符。在打印了前一本书的交易金额之和后,我们将最近读入的交易记录的副本赋予 `sum`,并递增迭代器,这将读取下一条交易记录。循环会这样持续下去,直至遇到错误或文件尾。在退出之前,记住要打印输入中最后一本书的交易金额之和。

407

10.4.2 节练习

练习 10.29: 编写程序,使用流迭代器读取一个文本文件,存入一个 `vector` 中的 `string` 里。

练习 10.30: 使用流迭代器、`sort` 和 `copy` 从标准输入读取一个整数序列,将其排序,并将结果写到标准输出。

练习 10.31: 修改前一题的程序,使其只打印不重复的元素。你的程序应使用 `unique_copy` (参见 10.4.1 节,第 359 页)。

练习 10.32: 重写 1.6 节(第 21 页)中的书店程序,使用一个 `vector` 保存交易记录,使用不同算法完成处理。使用 `sort` 和 10.3.1 节(第 345 页)中的 `compareIsbn` 函数来排序交易记录,然后使用 `find` 和 `accumulate` 求和。

练习 10.33: 编写程序，接受三个参数：一个输入文件和两个输出文件的文件名。输入文件保存的应该是整数。使用 `istream_iterator` 读取输入文件。使用 `ostream_iterator` 将奇数写入第一个输出文件，每个值之后都跟一个空格。将偶数写入第二个输出文件，每个值都独占一行。

10.4.3 反向迭代器

反向迭代器就是在容器中从尾元素向首元素反向移动的迭代器。对于反向迭代器，递增（以及递减）操作的含义会颠倒过来。递增一个反向迭代器（`++it`）会移动到前一个元素；递减一个迭代器（`--it`）会移动到下一个元素。

除了 `forward_list` 之外，其他容器都支持反向迭代器。我们可以通过调用 `rbegin`、`rend`、`crbegin` 和 `crend` 成员函数来获得反向迭代器。这些成员函数返回指向容器尾元素和首元素之前一个位置的迭代器。与普通迭代器一样，反向迭代器也有 `const` 和非 `const` 版本。

图 10.1 显示了一个名为 `vec` 的假设的 `vector` 上的 4 种迭代器：

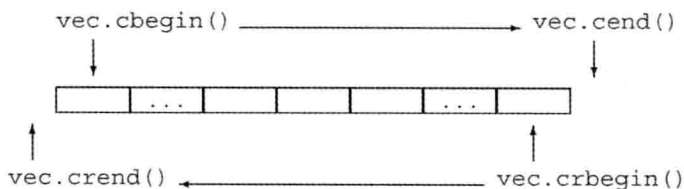


图 10.1: 比较 `cbegin/cend` 和 `crbegin/crend`

下面的循环是一个使用反向迭代器的例子，它按逆序打印 `vec` 中的元素：

```
vector<int> vec = {0,1,2,3,4,5,6,7,8,9};
// 从尾元素到首元素的反向迭代器
for (auto r_iter = vec.crbegin();    // 将 r_iter 绑定到尾元素
      r_iter != vec.crend();        // crend 指向首元素之前的位置
      ++r_iter)                    // 实际是递减，移动到前一个元素
    cout << *r_iter << endl;      // 打印 9, 8, 7, ... 0
```

408

虽然颠倒递增和递减运算符的含义可能看起来令人混淆，但这样做使我们可以用算法透明地向前或向后处理容器。例如，可以通过向 `sort` 传递一对反向迭代器来将 `vector` 整理为递减序：

```
sort(vec.begin(), vec.end()); // 按“正常序”排序 vec
// 按逆序排序：将最小元素放在 vec 的末尾
sort(vec.rbegin(), vec.rend());
```

反向迭代器需要递减运算符

不必惊讶，我们只能从既支持 `++` 也支持 `--` 的迭代器来定义反向迭代器。毕竟反向迭代器的目的是在序列中反向移动。除了 `forward_list` 之外，标准容器上的其他迭代器都既支持递增运算又支持递减运算。但是，流迭代器不支持递减运算，因为不可能在一个流中反向移动。因此，不可能从一个 `forward_list` 或一个流迭代器创建反向迭代器。

反向迭代器和其他迭代器间的关系

假定有一个名为 `line` 的 `string`，保存着一个逗号分隔的单词列表，我们希望打印



line 中的第一个单词。使用 find 可以很容易地完成这一任务：

```
// 在一个逗号分隔的列表中查找第一个元素
auto comma = find(line.cbegin(), line.cend(), ',');
cout << string(line.cbegin(), comma) << endl;
```

如果 line 中有逗号，那么 comma 将指向这个逗号；否则，它将等于 line.cend()。当我们打印从 line.cbegin() 到 comma 之间的内容时，将打印到逗号为止的字符，或者打印整个 string（如果其中不含逗号的话）。

如果希望打印最后一个单词，可以改用反向迭代器：

```
// 在一个逗号分隔的列表中查找最后一个元素
auto rcomma = find(line.crbegin(), line.crend(), ',');
```

由于我们将 crbegin() 和 crend() 传递给 find，find 将从 line 的最后一个字符开始向前搜索。当 find 完成后，如果 line 中有逗号，则 rcomma 指向最后一个逗号——即，它指向反向搜索中找到的第一个逗号。如果 line 中没有逗号，则 rcomma 指向 line.crend()。

当我们试图打印找到的单词时，最有趣的部分就来了。看起来下面的代码是显然的方法

```
// 错误：将逆序输出单词的字符
cout << string(line.crbegin(), rcomma) << endl;
```

409 ▶ 但它会生成错误的输出结果。例如，如果我们的输入是

FIRST,MIDDLE, LAST

则这条语句会打印 TSAL！

图 10.2 说明了问题所在：我们使用的是反向迭代器，会反向处理 string。因此，上述输出语句从 crbegin 开始反向打印 line 中内容。而我们希望按正常顺序打印从 rcomma 开始到 line 末尾间的字符。但是，我们不能直接使用 rcomma。因为它是一个反向迭代器，意味着它会反向朝着 string 的开始位置移动。需要做的是，将 rcomma 转换回一个普通迭代器，能在 line 中正向移动。我们通过调用 reverse_iterator 的 base 成员函数来完成这一转换，此成员函数会返回其对应的普通迭代器：

```
//正确：得到一个正向迭代器，从逗号开始读取字符直到 line 末尾
cout << string(rcomma.base(), line.cend()) << endl;
```

给定和之前一样的输入，这条语句会如我们的预期打印出 LAST。

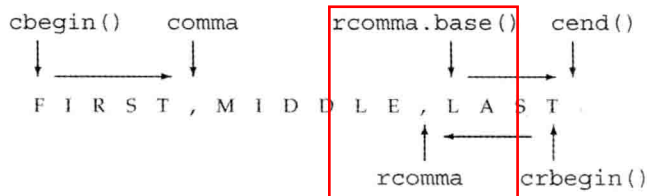


图 10.2：反向迭代器和普通迭代器间的关系

图 10.2 中的对象显示了普通迭代器与反向迭代器之间的关系。例如，rcomma 和 rcomma.base() 指向不同的元素，line.crbegin 和 line.cend() 也是如此。这些不同保证了元素范围无论是正向处理还是反向处理都是相同的。

从技术上讲，普通迭代器与反向迭代器的关系反映了左闭合区间（参见 9.2.1 节，第 296 页）的特性。关键点在于 `[line.crbegin(), rcomma)` 和 `[rcomma.base(), line.cend())` 指向 `line` 中相同的元素范围。为了实现这一点，`rcomma` 和 `rcomma.base()` 必须生成相邻位置而不是相同位置，`crbegin()` 和 `cend()` 也是如此。



反向迭代器的目的是表示元素范围，而这些范围是不对称的，这导致一个重要的结果：当我们从一个普通迭代器初始化一个反向迭代器，或是给一个反向迭代器赋值时，结果迭代器与原迭代器指向的并不是相同的元素。

10.4.3 节练习

练习 10.34: 使用 `reverse_iterator` 逆序打印一个 `vector`。

练习 10.35: 使用普通迭代器逆序打印一个 `vector`。

练习 10.36: 使用 `find` 在一个 `int` 的 `list` 中查找最后一个值为 0 的元素。

练习 10.37: 给定一个包含 10 个元素的 `vector`，将位置 3 到 7 之间的元素按逆序拷贝到一个 `list` 中。

10.5 泛型算法结构



任何算法的最基本的特性是它要求其迭代器提供哪些操作。某些算法，如 `find`，只要求通过迭代器访问元素、递增迭代器以及比较两个迭代器是否相等这些能力。其他一些算法，如 `sort`，还要求读、写和随机访问元素的能力。算法所要求的迭代器操作可以分为 5 个迭代器类别（`iterator category`），如表 10.5 所示。每个算法都会对它的每个迭代器参数指明须提供哪类迭代器。

表 10.5: 迭代器类别		
输入迭代器	只读，不写；单遍扫描，只能递增	输入输出，单扫只增 可读可写，多扫只增 迭代器层次逐渐提高， 高层次的可以兼容低层次的
输出迭代器	只写，不读；单遍扫描，只能递增	
前向迭代器	可读写；多遍扫描，只能递增	
双向迭代器	可读写；多遍扫描，可递增递减	
随机访问迭代器	可读写，多遍扫描，支持全部迭代器运算	

第二种算法分类的方式（如我们在本章开始所做的）是按照是否读、写或是重排序列中的元素来分类。附录 A 按这种分类方法列出了所有算法。

算法还共享一组参数传递规范和一组命名规范，我们在介绍迭代器类别之后将介绍这些内容。

10.5.1 5 类迭代器



类似容器，迭代器也定义了一组公共操作。一些操作所有迭代器都支持，另外一些只有特定类别的迭代器才支持。例如，`ostream_iterator` 只支持递增、解引用和赋值。`vector`、`string` 和 `deque` 的迭代器除了这些操作外，还支持递减、关系和算术运算。

迭代器是按它们所提供的操作来分类的，而这种分类形成了一种层次。除了输出迭代器